صف جا باز شود.

- **ظرفیت نامحدود**[1]: طول صف به صورت بالقوه نامتناهی است؛ بنابراین، پیغام‌ها به هر تعداد می‌توانند در آن منتظر شوند. فرستنده هرگز مسدود نمی‌شود.

```
message next_produced;

while (true) {
        /* produce an item in next_produced */

        send(next_produced);
}
```

شکل ۳-۱۴ پردازه‌ی تولیدکننده با استفاده از عبور پیغام.

```
message next_consumed;

while (true) {
        receive(next_consumed);

        /* consume the item in next_consumed */
}
```

شکل ۳-۱۵ پردازه‌ی مصرف‌کننده با استفاده از عبور پیغام.

۳-۷ مثال‌هایی از سیستم‌های IPC

در این بخش، چهار سیستم متفاوت IPC را بررسی می‌کنیم. در ابتدا API پوزیکس را برای حافظه‌ی مشترک مطرح کرده و سپس به بحث عبور پیغام در سیستم‌عامل Mach می‌پردازیم. در ادامه، IPC ویندوز را مطرح می‌کنیم که به طرز جالبی از حافظه‌ی مشترک به عنوان راهکاری برای فراهم‌نمودن انواع خاصی از عبور پیغام استفاده می‌کند. ما با لوله‌ها جمع‌بندی می‌کنیم که یکی از راهکارهای IPC نخستین در سیستم‌های یونیکس می‌باشد.

۳-۷-۱ حافظه‌ی مشترک پوزیکس

چندین راهکار IPC برای سیستم‌های پوزیکس موجودند که از آن جمله می‌توان به حافظه‌ی مشترک و عبور پیغام اشاره کرد. در اینجا، به حافظه‌ی مشترک API پوزیکس می‌پردازیم.

حافظه‌ی مشترک پوزیکس با استفاده از فایل‌های نگاشته به حافظه که ناحیه‌ی حافظه‌ی مشترک را به یک فایل مرتبط می‌کند، سازمان‌دهی می‌شود. هر پردازه در ابتدا بایستی با استفاده از فراخوانی سیستمی shm_open()، یک شئ حافظه‌ی مشترک را به صورت زیر ایجاد کند:

```
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

1. Unbounded capacity

پارامتر اول، نام شئ حافظه‌ی مشترک را مشخص می‌کند. پردازه‌هایی که به دسترسی این شئ حافظه‌ی مشترک تمایل دارند بایستی با این نام به شئ ارجاع بدهند. پارامترهای بعدی مشخص می‌کنند که در صورتی که شئ هنوز ایجاد نشده است ایجاد شود (O_CREAT) و این که شئ برای خواندن و نوشتن باز شود (O_RDWR). پارامتر آخری، مجوزهای دسترسی فایل شئ حافظه‌ی مشترک را برپا می‌کند. فراخوانی موفقیت‌آمیز shm_open()، یک توصیف‌گر فایل صحیح برای شئ حافظه‌ی مشترک بر می‌گرداند.

همین که شئ بنا می‌شود، تابع ftruncate() برای پیکربندی اندازه‌ی شئ به بایت به‌کار گرفته می‌شود. فراخوانی

```
ftruncate(shm_fd, 4096);
```

اندازه‌ی شئ را برابر ۴۰۹۶ بایت انتخاب می‌کند.

در نهایت، تابع mmap()، یک فایل نگاشته به حافظه، شامل شئ حافظه‌ی مشترک برپا می‌کند. لذا، یک اشاره‌گر به فایل نگاشته به حافظه بر می‌گرداند که برای دسترسی شئ حافظه‌ی مشترک استفاده می‌شود.

برنامه‌های نشان داده شده در شکل ۳–۱۶ و ۳–۱۷ برای پیاده‌سازی حافظه‌ی مشترک از مدل تولیدکننده–مصرف‌کننده استفاده می‌کنند. تولیدکننده یک شئ حافظه‌ی مشترک را برپا کرده و در حافظه‌ی مشترک می‌نویسد و مصرف‌کننده از حافظه‌ی مشترک می‌خواند.

تولیدکننده‌ای که در شکل ۳–۱۶ نشان داده می‌شود، یک شئ حافظه‌ی مشترک به نام OS را ایجاد کرده و رشته‌ی مشهور «Hello World» را در حافظه‌ی مشترک می‌نویسد. برنامه، شئ حافظه‌ی مشترک را با اندازه‌ی مشخص شده به حافظه نگاشت می‌دهد و امکان نوشتن شئ را فراهم می‌کند. (پر واضح است که نوشتن فقط برای تولیدکننده لازم است.) پرچم MAP_SHARED مشخص می‌کند که تغییرات شئ حافظه‌ی مشترک برای تمامی پردازه‌هایی که شئ را به اشتراک می‌گیرند، قابل رؤیت خواهد بود. توجه داشته باشید که شئ حافظه‌ی مشترک را با فراخوانی تابع sprintf() و نوشتن رشته‌ی قالب‌بندی شده در اشاره‌گر ptr می‌نویسیم. بعد از هر بار نوشتن، بایستی اشاره‌گر را به تعداد بایت‌های نوشته شده افزایش دهیم.

پردازه‌ی مصرف‌کننده‌ای که در شکل ۳–۱۷ نشان داده می‌شود، محتوای حافظه‌ی مشترک را خوانده و خروجی می‌نماید. مصرف‌کننده تابع shm_unlink() را نیز احضار می‌کند که سگمنت حافظه‌ی مشترک را بعد از این که مصرف‌کننده دسترسی کرد، بر می‌دارد. تمرین‌های بیشتری را که از API حافظه‌ی مشترک پوزیکس استفاده می‌کنند در تمرین‌های برنامه‌نویسی انتهای همین فصل تدارک دیده‌ایم. به علاوه، پوشش جزئیات بیشتری از نگاشت حافظه را در بخش ۱۳–۵ فراهم می‌کنیم.

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main() {
        /* the size (in bytes) of shared memory object */
        const int SIZE = 4096;
        /* name of the shared memory object */
        const char *name = "OS";
        /* strings written to shared memory */
        const char *message_0 = "Hello";
        const char *message_1 = "World!";
        /* shared memory file descriptor */
        int shm_fd;
        /* pointer to shared memory object */
        void *ptr;
        /* create the shared memory object */
        shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
        /* configure the size of the shared memory object */
        ftruncate(shm_fd, SIZE);
        /* memory map the shared memory object */
        ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
        /* write to the shared memory object */
        sprintf(ptr,"%s",message_0);
        ptr += strlen(message_0);
        sprintf(ptr,"%s",message_1);
        ptr += strlen(message_1);
        return 0;
}
```

شکل ۳-۱۶ پردازه‌ی تولیدکننده که API حافظه‌ی مشترک پوزیکس را روشن می‌کند.

۳-۷-۲ عبور پیغام Mach

در ادامه، به عنوان مثالی از عبور پیغام، سیستم‌عامل Mach را مطرح می‌کنیم. Mach مخصوصاً برای سیستم‌های توزیع‌شده طراحی شد، اما مشخص شد که برای سیستم‌های موبایل و رومیزی نیز مناسب است، شاهد این مطلب، شمول آن در سیستم‌عامل‌های macOS و iOS است که در فصل ۲ مورد بحث قرار گرفت.

هسته‌ی Mach از ایجاد و حذف وظایف متعددی پشتیبانی می‌کند که شبیه پردازه‌ها هستند اما نخ‌های کنترل متعدد و منابع مرتبط معدودی دارند. عمده‌ی ارتباطات Mach—از جمله تمامی اطلاعات بین وظایف—توسط **پیغام‌ها** انجام می‌شوند. پیغام‌ها به (یا از) صندوق‌های پستی ارسال (یا دریافت) می‌شوند که در Mach، **پورت** نامیده می‌شوند. درگاه‌ها در اندازه متناهی بوده و

یک‌طرفه هستند؛ برای ارتباط دوطرفه، یک پیغام به یک پورت فرستاده می‌شود و یک پاسخ به یک پورت **پاسخ**[1] مجزا فرستاده می‌شود. هر پورت می‌توانند چندین فرستنده داشته باشد، اما تنها یک دریافت‌کننده دارد. Mach از درگاه‌ها برای نمایش منابعی همچون وظایف، نخ‌ها، حافظه، و پردازنده‌ها استفاده می‌کند، در حالی که عبور پیغام یک رویکرد شئ‌گرایی برای تعامل با این منابع سیستمی و سرویس‌ها فراهم می‌کند. عبور پیغام ممکن است بین هر دو پورت بر روی میزبان یکسان یا بر روی میزبان‌های مجزا در یک سیستم توزیع‌شده اتفاق بیفتد.

```
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main()
{
        /* the size (in bytes) of shared memory object */
        const int SIZE = 4096;
        /* name of the shared memory object */
        const char *name = "OS";
        /* shared memory file descriptor */
        int shm_fd;
        /* pointer to shared memory obect */
        void *ptr;

        /* open the shared memory object */
        shm_fd = shm_open(name, O_RDONLY, 0666);

        /* memory map the shared memory object */
        ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

        /* read from the shared memory object */
        printf("%s",(char *)ptr);

        /* remove the shared memory object */
        shm_unlink(name);

        return 0;
}
```

شکل ۳-۱۷ پردازه‌ی مصرف‌کننده که API حافظه‌ی مشترک پوزیکس را روشن می‌کند.

مرتبط با هر پورت، یک مجموعه **حقوق پورت** است که قابلیت‌های ضروری برای یک وظیفه به‌منظور تعامل با پورت را مشخص می‌کنند. برای مثال، برای این که یک وظیفه پیغامی را از یک

1. Reply

پورت دریافت کند، بایستی قابلیت MACH_PORT_RIGHT_RECEIVE آن پورت را داشته باشد. وظیفه‌ای که یک پورت را ایجاد می‌کند مالک پورت است، و مالک تنها وظیفه‌ای است که اجازه دارد تا پیغام‌هایی را از آن پورت دریافت کند. مالک یک پورت ممکن است قابلیت‌های یک پورت را نیز دستکاری کند. این کار اکثراً در برپایی یک پورت پاسخ انجام می‌شود. برای مثال، فرض کنید که وظیفه‌ی T1 مالک پورت P1 است، و پیغامی به پورت P2 ارسال می‌کند که تحت مالکیت وظیفه‌ی T2 می‌باشد. اگر T1 انتظار داشته باشد که از T2 پاسخی دریافت کند، بایستی به T2 حق MACH_PORT_RIGHT_SEND پورت P1 را اعطا نماید. مالکیت حقوق پورت در سطح وظیفه است، بدین معنی که تمامی نخ‌های متعلق به وظیفه‌ای یکسان، حقوق پورت یکسانی را به‌اشتراک می‌گیرند. بنابراین، دو نخ متعلق به وظیفه‌ای یکسان می‌توانند به‌راحتی از طریق پورت نخ مرتبط با هر نخ، تبادل پیغام داشته باشند.

موقعی که وظیفه‌ای ایجاد می‌شود، دو پورت خاص—پورت **Task Self** و پورت **اطلاع‌رسانی**[1]—نیز ایجاد می‌شوند. هسته حقوق دریافت از پورت Task Self را دارد که به یک وظیفه امکان می‌دهد تا به هسته پیغام‌هایی ارسال کند. هسته می‌تواند اطلاعیه‌های رخداد رویدادها را به پورت اطلاع‌رسانی (البته که وظیفه برای آن حقوق دریافت دارد) یک وظیفه ارسال کند.

فراخوانی تابع mach_port_allocate() یک پورت جدید ایجاد کرده و برای صف پیغام‌های آن فضا تخصیص می‌دهد. آن حقوق پورت را نیز مشخص می‌کند. هر حق پورت معرف یک **نام** برای آن پورت است و یک پورت تنها می‌تواند از طریق یک حق مورد دسترسی قرار بگیرد. نام درگاه‌ها مقادیر عددی صحیحی هستند و مثل توصیف‌گرهای فایل یونیکس رفتار می‌کنند. مثال زیر ایجاد یک پورت را با استفاده از این API نمایش می‌دهد:

```
mach_port_t port; // the name of the port right

mach_port_allocate(
               mach_task_self(), // a task referring to itself
               MACH_PORT_RIGHT_RECEIVE, // the right for this port
               &port); // the name of the port right
```

هر وظیفه به یک **پورت بالاآورنده**[2] نیز دسترسی دارد که به وظیفه امکان می‌دهد تا یک پورتی را که ایجاد کرده است با **کارگزار بالاآورنده‌ی**[3] سطح سیستم ثبت کند. همین که یک پورت با کارگزار بالاآورنده ثبت شد، سایر وظیفه‌ها می‌توانند پورت را در رجیستری جویا شده و حقوقی برای ارسال پیغام به پورت بدست آورند.

صف مرتبط با هر پورت در اندازه متناهی بوده و در آغاز خالی است. همچنان‌که پیغام‌ها به پورت ارسال می‌شوند، پیغام‌ها به صف کپی می‌شوند. تمامی پیغام‌ها با اطمینان دریافت شده و الویت یکسانی دارند. Mach تضمین می‌کند که پیغام‌های متعددی که از

1. Notify
2. Bootstrap port
3. Bootstrap server

فرستنده‌ای یکسان می‌آیند به‌ترتیب اول-ورود، اول‌خروج (FIFO) در صف قرار می‌گیرند اما یک ترتیب مطلقی را تضمین نمی‌دهد. برای نمونه، پیغام‌هایی که از دو فرستنده می‌رسند ممکن است به‌هر ترتیبی در صف قرار بگیرند.

پیغام‌های Mach شامل دو فیلد زیر هستند:

- یک سرآیند پیغام با اندازه‌ی ثابت که شامل ابرداده‌هایی راجع به پیغام، اعم از اندازه‌ی پیغام در کنار درگاه‌های مبدأ و مقصد می‌باشد. مرسوم است که نخ فرستنده انتظار یک پاسخ را داشته باشد، لذا نام پورت مبدأ به وظیفه‌ی دریافت‌کننده ارسال می‌شود که می‌تواند از آن در ارسال یک پاسخ برای «آدرس برگشتی» استفاده کند.
- بدنه‌ای با اندازه‌ی متغیر که شامل داده‌هاست.

پیغام‌ها ممکن است **ساده** و یا **پیچیده** باشند. پیغام ساده شامل داده‌های معمولی و بدون ساختار کاربر است که توسط هسته تفسیر نمی‌شوند. پیغام پیچیده ممکن است شامل اشاره‌گرهایی به مکان‌هایی از حافظه داشته باشد که شامل داده‌هایی (تحت عنوان داده‌های «خارج از خط[1]») است یا ممکن است برای انتقال حقوق پورت به وظیفه‌ی دیگری استفاده شوند. اشاره‌گر داده‌های خارج از خط مخصوصاً موقعی که یک پیغام بایستی قالب‌های بزرگی از داده‌ها را ارسال کند مفید هستند. یک پیغام ساده مستلزم کپی و بسته‌بندی داده‌ها در پیغام خواهد بود؛ انتقال داده‌های خارج از خط تنها مستلزم اشاره‌گری است که به مکان حافظه‌ای که داده‌ها ذخیره شده‌اند ارجاع می‌دهد.

تابع mach_msg() API استاندارد برای هر دوی ارسال و دریافت پیغام‌هاست. مقدار یکی از پارامترهای تابع—MACH_SEND_MSG و یا MACH_RCV_MSG—نشان می‌دهد که آیا آن یک عمل ارسال یا دریافت است. ما اکنون چگونگی استفاده از آن را موقعی که یک وظیفه‌ی کارفرما پیغام ساده‌ای به یک وظیفه‌ی کارگزار ارسال می‌کند نشان می‌دهیم. فرض کنید که به‌ترتیب، دو پورت—client و server—مرتبط با وظایف کارفرما و کارگزار داریم. کد شکل ۳-۱۸ نشان می‌دهد که وظیفه‌ی کارفرما یک سرآیندی ساخته و پیغامی به کارگزار می‌فرستند، همین‌طور وظیفه‌ی کارگزار پیغام فرستاده شده از کارفرما را دریافت می‌کند.

فراخوانی تابع mach_msg() توسط برنامه‌های کاربر برای انجام عبور پیغام احضار می‌شود. سپس، mach_msg() تابع mach_msg_trap() را احضار می‌کند که یک فراخوانی سیستمی به هسته‌ی Mach می‌باشد. در ادامه، mach_msg_trap() در میان هسته، تابع mach_msg_overwrite_trap() را فراخوانی می‌کند که در ادامه به ارسال واقعی پیغام رسیدگی می‌کند.

عملیات ارسال و دریافت خودشان انعطاف‌پذیر هستند. برای نمونه، موقعی که پیغام به پورتی ارسال می‌شود، صف آن ممکن است پر باشد. اگر صف پر نباشد، پیغام به صف کپی شده و وظیفه‌ی فرستنده ادامه می‌یابد. اگر صف پورت پر باشد، فرستنده چندین انتخاب دارد (که از طریق پارامترهای mach_msg() مشخص می‌شوند):

1. Out-of-line

۱- به‌صورت نامتناهی منتظر بماند تا این که جایی در صف پیدا شود.
۲- حداکثر n میلی‌ثانیه منتظر بماند.
۳- اصلاً منتظر نماند بلکه بلافاصله برگردد.
۴- پیغام را به‌صورت موقت کَش کند. در اینجا، پیغام به سیستم‌عامل داده می‌شود تا نگه بدارد، حتی اگر صفی که پیغام به آن ارسال می‌شود پر باشد. موقعی که پیغام می‌تواند در صف گذاشته شود، دوباره یک پیغام اطلاع‌رسانی به فرستنده ارسال می‌شود. تنها یک پیغام به یک صف پر می‌تواند در هر لحظه برای یک نخ فرستنده‌ی مشخصی به‌تعویق بیفتد.

```
#include<mach/mach.h>

struct message {
        mach msg_header_t header;
        int data;
};
mach_port_t client;
mach_port_t server;

/* Client Code */
struct message message;
// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;
// send the message
mach_msg(&message.header, // message header
        MACH_SEND_MSG, // sending a message
        sizeof(message), // size of message sent
        0, // maximum size of received message - unnecessary
        MACH_PORT_NULL, // name of receive port - unnecessary
        MACH_MSG_TIMEOUT_NONE, // no time outs
        MACH_PORT_NULL // no notify port
);
/* Server Code */
struct message message;
// receive the message
mach_msg(&message.header, // message header
        MACH_RCV_MSG, // sending a message
        0, // size of message sent
        sizeof(message), // maximum size of received message
        server, // name of receive port
        MACH_MSG_TIMEOUT_NONE, // no time outs
        MACH_PORT_NULL // no notify port
```

```
) ;
```

شکل ۳-۱۸ برنامه‌ی نمونه‌ای که عبور پیغام را در Mach نشان می‌دهد.

گزینه‌ی آخر در رابطه با وظایف کارگزار است. وظیفه‌ی کارگزار بعد از پایان یک درخواست، نیاز دارد که پاسخی یک‌باره به وظیفه‌ی درخواست‌کننده‌ی سرویس ارسال کند، اما بایستی به سرویس درخواست‌های دیگر ادامه بدهد، حتی اگر پورت پاسخ کارفرما پر باشد.

مسأله‌ی اصلی که کلاً در سیستم‌های پیغام وجود دارد، کارایی ضعیفی است که از کپی مضاعف پیغام‌ها از پورت فرستنده در پورت گیرنده ناشی می‌شود. سیستم پیغام Mach در تلاش است تا با استفاده از تکنیک‌های مدیریت حافظه‌ی مجازی (فصل ۱۰) از عملیات کپی اجتناب کند. Mach در اصل، فضای آدرس مربوط به پیغام فرستنده را به فضای آدرس گیرنده نگاشت می‌دهد. بنابراین، خود پیغام به واقع هرگز کپی نمی‌شود، چون هر دوی فرستنده و گیرنده حافظه‌ی مشترکی را دست‌یابی می‌کنند. این تکنیک مدیریت پیغام، کمک زیادی به بالارفتن کارایی می‌نماید، اما کارکرد آن فقط در پیغام‌های داخلی سیستم است.

۳-۷-۳ ویندوز

سیستم‌عامل ویندوز، مثالی از یک طراحی مدرن است که برای افزایش کارکرد و کاهش زمان لازم برای پیاده‌سازی امکانات جدید، ماژولاریتی را بکار می‌برد. ویندوز، پشتیبانی محیط‌های عملیاتی متعدد یا *زیرسیستم‌ها* را فراهم می‌کند. ارتباط برنامه‌های کاربردی با این زیرسیستم‌ها از طریق یک راهکار عبور پیغام صورت می‌گیرد. بنابراین، می‌توان برنامه‌های کاربردی را کارفرماهای یک کارگزار زیرسیستم در نظر گرفت.

امکان عبور پیغام ویندوز، امکان **فراخوانی روال محلی پیشرفته**[1] (**ALPC**) نامیده می‌شود که برای ارتباط میان دو پردازه روی ماشین یکسان استفاده می‌شود. آن مشابه راهکار معمول فراخوانی روال دور (RPC) است که به صورت گسترده استفاده می‌شود، اما بهینه شده و خاص ویندوز می‌باشد. (فراخوانی‌های روال دور با جزئیات بیشتر در بخش ۳-۸-۲ پوشش داده می‌شوند.) ویندوز هم مثل Mach، برای برپایی و حفظ یک اتصال میان دو پردازه از یک شئ درگاه استفاده می‌کند. ویندوز از دو نوع درگاه استفاده می‌کند که عبارتند از: **درگاه‌های اتصال**[2] و **درگاه‌های ارتباط**[3].

پردازه‌های کارگزار، اشیاء درگاه اتصال را منتشر می‌کنند که برای تمامی پردازه‌ها قابل رؤیت هستند. موقعی که یک کارفرما سرویس‌هایی را از یک زیرسیستم می‌خواهد، دستگیره‌ای به شئ درگاه اتصال کارگزار باز کرده و یک درخواست اتصال به آن درگاه می‌فرستند. سپس، کارگزار یک کانال ایجاد کرده و یک دستگیره به کارفرما بر می‌گرداند. کانال، حاوی یک جفت درگاه ارتباط خصوصی است: یکی برای پیغام‌های کارفرما–کارگزار، دیگری برای پیغام‌های

1. Advanced local procedure call
2. Connection ports
3. Communication ports